GENIE FOR EXTERNAL USERS · OAUTH CUSTOM IDENTITY CLAIMS

Implementation Guide

Community reference, not official Databricks documentation. OAuth custom identity claims are announced on the Databricks *What's coming?* page; check your workspace with `sql/check_availability.sql`.

This guide builds the pattern from the Overview end to end, using the two fictional companies in this repo: **Northwind Retirement Services**, a 401(k) recordkeeper, and **Larkspur Pharmacy Benefits**, a pharmacy benefit manager. Every SQL block below is the real file the bundle deploys, and every trace card and result table comes from a live run of the isolation suite.

Before you start

- A Unity Catalog workspace and a serverless or pro SQL warehouse.
- A workspace admin for the one-time bootstrap step (it creates service principals and their OAuth secrets).
- The `CREATE CATALOG` privilege, or an existing catalog you can create schemas in.
- The Databricks CLI with bundle support, and Python 3.10 or later for the local app and the command-line tools.

Step 1: Check availability

OAuth custom identity claims are announced on the Databricks *What's coming?* page (AWS, Azure). There is no workspace setting to turn on. Three checks tell you whether your workspace has them.

Check 1 and 2, in the SQL editor. These are in `sql/check_availability.sql`.

```
DESCRIBE FUNCTION current_oauth_custom_identity_claim;
SELECT current_oauth_custom_identity_claim() AS claim;
```

Result	Meaning
`DESCRIBE` returns a description	The function exists in this workspace
`SELECT` returns `NULL`	Available. Claim-keyed views can be created by any identity.

Result	Meaning
`SELECT` raises `OAUTH_CUSTOM_IDENTITY_CLAIM_NOT_PROVIDED`	Available. Claim-keyed views must be created from a session that carries a claim; the bundle does this for you.
Either raises `UNRESOLVED_ROUTINE`	Not available in this workspace yet

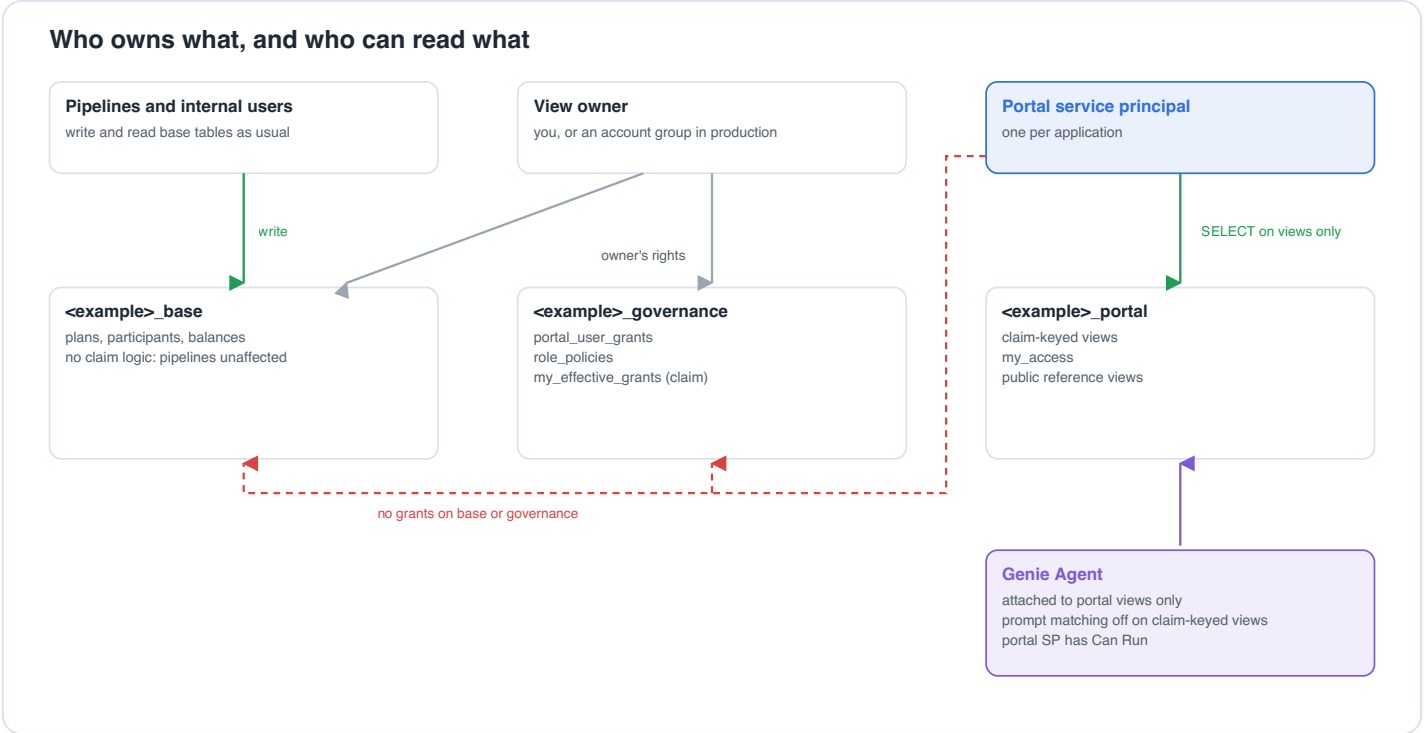
Check 3, with a service principal token. Request a token with `custom_claim` set, then read it back through the Statement Execution API:

```
TOKEN=$(curl -s -X POST "https://<workspace-host>/oidc/v1/token" \
  -u "<client-id>:<client-secret>" \
  -d grant_type=client_credentials -d scope=all-apis \
  -d custom_claim=availability-probe | python3 -c "import json,sys; print(json.load(sys.stdin)
['access_token'])")

curl -s -X POST "https://<workspace-host>/api/2.0/sql/statements" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"warehouse_id": "<warehouse-id>", "wait_timeout": "30s",
       "statement": "SELECT current_oauth_custom_identity_claim() AS claim, session_user() AS
caller"}'
```

The result must contain `availability-probe` and the service principal's application id. If the token request returns HTTP 403 with `custom_claim is not enabled for account`, ask your Databricks account team. Notebook 01 and `python -m claims_kit.availability` run all three checks and report which deploy mode the workspace needs.

Architecture



Who owns what, and who can read what.

Identity	May do
Pipelines and internal users	Write and read the base tables as usual. Base tables carry no claim logic.
View owner (you, or an account group in production)	Owns the claim-keyed views. Views read the base and governance tables with the owner's rights.
Portal service principal, one per application	Reads through its application's account group: `USE CATALOG`, `USE SCHEMA` on its portal schema, `SELECT` on its portal views. Nothing else.
Row policies (layered mode)	ABAC row filter policies on every base table the portal views read, for the application's group only. Pipelines, administrators and the view owner are unaffected.
Genie Agent	Attached to the portal views only. The portal service principal has Can Run.
Operators	Change entitlements with their own identity, through your admin tooling (the local app's admin tab in this repo).

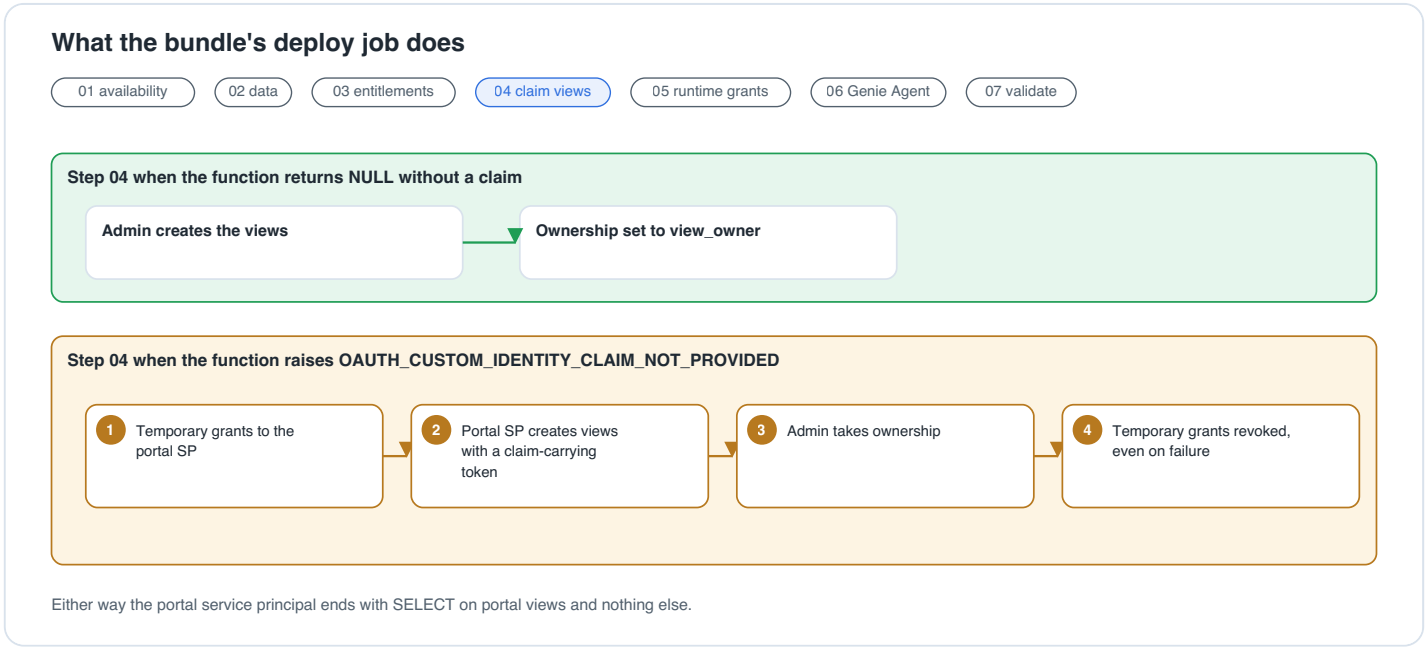
Step 2: Deploy with the bundle

```
databricks bundle deploy -p <profile> --var warehouse_id=<warehouse-id>
databricks bundle run bootstrap -p <profile> --var warehouse_id=<warehouse-id>
databricks bundle run northwind_deploy -p <profile> --var warehouse_id=<warehouse-id>
databricks bundle run larkspur_deploy -p <profile> --var warehouse_id=<warehouse-id>
databricks bundle run northwind_drills -p <profile> --var warehouse_id=<warehouse-id>   # optional, test deployments only
```

The layered model needs the two governed tags `portal_grain` and `portal_key` (an account admin creates them; see the Admin Runbook). To evaluate without them, add `--var tag_set=stand_in` to every command.

Variable	Default	Meaning
`catalog`	`genie_claims_ref`	Catalog for both examples
`create_catalog`	`true`	Create the catalog if it is missing. Use `false` with an existing catalog.
`warehouse_id`	(required)	SQL warehouse for the service principals, the Statement API and Genie
`secret_scope`	`genie-claims-ref`	Where bootstrap stores each service principal's client id and secret
`view_owner`	the deploying user	Owner of the claim-keyed views. Use an account group in production.
`enforcement`	`layered`	`layered` adds ABAC row policies under the claim-keyed views. `views` deploys the views alone.
`tag_set`	`governed`	`governed` uses the custom governed tags. `stand_in` uses platform system tags, for evaluation only.

If you can't create catalogs, point the bundle at one you can use: `--var catalog=<existing catalog> --var create_catalog=false`.



Step 04 adapts to how the workspace treats a missing claim.

Notebook	What it does
`00_bootstrap_service_principals`	One service principal per application, an OAuth secret in the secret scope, Can Use on the warehouse, an account group per application, and the governed tags when permitted
`01_check_availability`	The three checks above, and the deploy mode
`02_create_data`	Catalog (optional), three schemas per example, deterministic synthetic data
`03_entitlements`	Per-user grants and per-role policies, with CHECK constraints, and the derived tables (policy inputs, precomputed spend cells)
`04_claim_views`	The claim-keyed views, in either deploy mode
`05_row_policies`	Tags, row-filter functions, row policies and default-deny, with a coverage check (layered mode)
`06_runtime_grants`	`SELECT` on portal views for the application's group, nothing else
`07_genie_agent`	The Genie Agent over the portal views, with Can Run for the service principal
`08_validate_isolation`	The isolation suite; the task fails if any check fails
`09_misconfiguration_drills`	The drills, in their own job; never part of deploy

Step 3: Understand each object (Northwind)

Three schemas per application keep pipelines, policy and the portal apart: `northwind_base`, `northwind_governance` and `northwind_portal`.

Entitlements

Per-user grants and per-role policies, with constraints that reject malformed claims and unknown roles:

```
-- Per-user entitlements. One row per user, role and scope; a user with two roles has two rows.
CREATE OR REPLACE TABLE `{{catalog}}`.`{{gov}}`.portal_user_grants (
  portal_user_id STRING    NOT NULL COMMENT 'Opaque application user id. This exact value travels
in the custom claim.',
  role           STRING    NOT NULL COMMENT 'participant, sponsor or advisor. Joins to
role_policies.',
  scope_type     STRING    NOT NULL COMMENT 'participant (one person) or plan (a whole plan)',
  scope_id       STRING    NOT NULL COMMENT 'A participant_id when scope_type is participant, a
plan_id when it is plan',
  valid_from     DATE      NOT NULL COMMENT 'First day the grant applies',
  valid_to       DATE               COMMENT 'Last day the grant applies. NULL means open-ended.',
  granted_by     STRING             COMMENT 'Who created the grant: seed, admin console or a
provisioning job',
  granted_at     TIMESTAMP          COMMENT 'When the grant was created'
) CLUSTER BY (portal_user_id) COMMENT 'Per-user entitlements for the Northwind portal.';

-- regexp_extract instead of RLIKE '^...$': in Java regex `$` also matches before a trailing
newline.
ALTER TABLE `{{catalog}}`.`{{gov}}`.portal_user_grants ADD CONSTRAINT claim_format CHECK
(regexp_extract(portal_user_id, '^[a-z0-9][a-z0-9._:-]{0,127}', 0) = portal_user_id);
ALTER TABLE `{{catalog}}`.`{{gov}}`.portal_user_grants ADD CONSTRAINT known_role CHECK (role IN
('participant', 'sponsor', 'advisor'));
ALTER TABLE `{{catalog}}`.`{{gov}}`.portal_user_grants ADD CONSTRAINT known_scope CHECK
(scope_type IN ('participant', 'plan'));
ALTER TABLE `{{catalog}}`.`{{gov}}`.portal_user_grants ADD CONSTRAINT valid_window CHECK
(valid_to IS NULL OR valid_to >= valid_from);

-- Per-role entitlements. Change a row here and every user with that role changes on their next
question.
CREATE OR REPLACE TABLE `{{catalog}}`.`{{gov}}`.role_policies (
  role                   STRING  NOT NULL COMMENT 'Role name used in portal_user_grants',
  can_view_own_accounts  BOOLEAN NOT NULL COMMENT 'May see balances and contributions of the
participant in scope',
  can_view_plan_aggregates BOOLEAN NOT NULL COMMENT 'May see plan totals for plans in scope',
  can_view_roster        BOOLEAN NOT NULL COMMENT 'May list the participants of plans in scope',
```

```
  mask_dob                BOOLEAN NOT NULL COMMENT 'Hide date of birth in the roster'
) COMMENT 'Per-role entitlements for the Northwind portal.';
```

The seed rows. Linda holds two roles, so she has two rows:

```
INSERT INTO `{{catalog}}`.`{{gov}}`.role_policies VALUES
  ('participant', true,  false, false, false),
  ('sponsor',     false, true,  true,  false),
  ('advisor',     false, true,  true,  true);

INSERT INTO `{{catalog}}`.`{{gov}}`.portal_user_grants VALUES
  ('pu-1001', 'participant', 'participant', 'P-1001', DATE '2026-01-01', NULL, 'seed',
current_timestamp()),
  ('pu-1002', 'participant', 'participant', 'P-1002', DATE '2026-01-01', NULL, 'seed',
current_timestamp()),
  ('pu-2001', 'sponsor',     'plan',        'PLAN-A', DATE '2026-01-01', NULL, 'seed',
current_timestamp()),
  ('pu-2001', 'participant', 'participant', 'P-2001', DATE '2026-01-01', NULL, 'seed',
current_timestamp()),
  ('pu-3001', 'advisor',     'plan',        'PLAN-A', DATE '2026-01-01', NULL, 'seed',
current_timestamp()),
  ('pu-3001', 'advisor',     'plan',        'PLAN-C', DATE '2026-01-01', NULL, 'seed',
current_timestamp());
```

The policy view

This is the only object that reads the claim. A grant counts only when it belongs to the user named in the token, the caller is this application's service principal, today is inside the grant's dates, and the role has a policy.

```
-- The one place where the custom claim is read. Every portal view joins to this view.
-- A grant counts only when ALL of these hold:
--   1. it belongs to the user named in the signed token     current_oauth_custom_identity_claim()
--   2. the caller is this application's service principal    session_user()
--   3. today is inside the grant's validity window
--   4. the grant's role has a policy row
-- A missing, unknown or malformed claim matches no grant, so every portal view returns no rows.
CREATE VIEW `{{catalog}}`.`{{gov}}`.my_effective_grants
COMMENT
'Active grants of the signed-in portal user, joined to their role policy. Answers only for the
Northwind portal service principal.'
AS
SELECT
  g.portal_user_id, g.role, g.scope_type, g.scope_id, g.valid_from, g.valid_to,
  p.can_view_own_accounts, p.can_view_plan_aggregates, p.can_view_roster, p.mask_dob
FROM `{{catalog}}`.`{{gov}}`.portal_user_grants AS g
```

```
JOIN `{{catalog}}`.`{{gov}}`.role_policies AS p ON p.role = g.role
WHERE g.portal_user_id = current_oauth_custom_identity_claim()
  AND session_user() = '{{portal_sp}}'
  AND current_date() >= g.valid_from
  AND (g.valid_to IS NULL OR current_date() <= g.valid_to);
```

The portal views

None of these mentions the claim. Each joins to `my_effective_grants`, groups the user's scope first so several grants never duplicate rows, and applies the role policy.

```
-- Portal views. None of them mentions the claim: they all join to governance.my_effective_grants.
-- Each scope is grouped first (one row per participant or plan), so a user with several matching
-- grants never sees duplicated rows. When grants disagree, the least restrictive one wins.

CREATE VIEW `{{catalog}}`.`{{portal}}`.my_access (
  role COMMENT 'Role of this grant: participant, sponsor or advisor',
  scope_type COMMENT 'participant or plan',
  scope_id COMMENT 'The participant or plan this grant covers',
  valid_from COMMENT 'First day of the grant',
  valid_to COMMENT 'Last day of the grant. NULL means open-ended.',
  can_view_own_accounts COMMENT 'May see own balances and contributions',
  can_view_plan_aggregates COMMENT 'May see plan totals',
  can_view_roster COMMENT 'May list plan participants',
  mask_dob COMMENT 'Date of birth is hidden in the roster'
)
COMMENT 'What the signed-in user is entitled to see. Use for questions like "what can I see?" or
"what is my role?".'
AS SELECT role, scope_type, scope_id, valid_from, valid_to, can_view_own_accounts,
can_view_plan_aggregates, can_view_roster, mask_dob
FROM `{{catalog}}`.`{{gov}}`.my_effective_grants;

CREATE VIEW `{{catalog}}`.`{{portal}}`.my_accounts (
  participant_id COMMENT 'Participant who owns the balance: the signed-in user',
  plan_id COMMENT 'Plan identifier: PLAN-A, PLAN-B or PLAN-C. Users call these Plan A, Plan B and
Plan C.',
  plan_name COMMENT 'Plan name (the sponsor plan name, not Plan A or Plan B)',
  fund_name COMMENT 'Fund the money is invested in',
  asset_class COMMENT 'Fund asset class',
  balance COMMENT 'Market value in US dollars',
  as_of_date COMMENT 'Valuation date'
)
COMMENT 'Balances of the signed-in participant, one row per fund. Use for "my balance" questions.
Empty for users who are not participants.'
AS
WITH my_participants AS (
  SELECT DISTINCT scope_id AS participant_id
```

```
  FROM `{{catalog}}`.`{{gov}}`.my_effective_grants
  WHERE scope_type = 'participant' AND can_view_own_accounts
)
SELECT b.participant_id, b.plan_id, pl.plan_name, f.fund_name, f.asset_class, b.balance,
b.as_of_date
FROM `{{catalog}}`.`{{base}}`.account_balances AS b
JOIN my_participants AS m ON m.participant_id = b.participant_id
JOIN `{{catalog}}`.`{{base}}`.plans AS pl ON pl.plan_id = b.plan_id
JOIN `{{catalog}}`.`{{base}}`.funds AS f ON f.fund_id = b.fund_id;

CREATE VIEW `{{catalog}}`.`{{portal}}`.my_contributions (
  participant_id COMMENT 'Participant the contribution belongs to: the signed-in user',
  plan_id COMMENT 'Plan identifier: PLAN-A, PLAN-B or PLAN-C. Users call these Plan A, Plan B and
Plan C.',
  plan_name COMMENT 'Plan name (the sponsor plan name, not Plan A or Plan B)',
  contribution_date COMMENT 'Payroll date',
  source COMMENT 'employee_pretax, employee_roth or employer_match',
  amount COMMENT 'Amount in US dollars'
)
COMMENT 'Payroll contributions of the signed-in participant in 2026. Empty for users who are not
participants.'
AS
WITH my_participants AS (
  SELECT DISTINCT scope_id AS participant_id
  FROM `{{catalog}}`.`{{gov}}`.my_effective_grants
  WHERE scope_type = 'participant' AND can_view_own_accounts
)
SELECT c.participant_id, c.plan_id, pl.plan_name, c.contribution_date, c.source, c.amount
FROM `{{catalog}}`.`{{base}}`.contributions AS c
JOIN my_participants AS m ON m.participant_id = c.participant_id
JOIN `{{catalog}}`.`{{base}}`.plans AS pl ON pl.plan_id = c.plan_id;

CREATE VIEW `{{catalog}}`.`{{portal}}`.plan_summary (
  plan_id COMMENT 'Plan identifier: PLAN-A, PLAN-B or PLAN-C. Users call these Plan A, Plan B and
Plan C.',
  plan_name COMMENT 'Plan name (the sponsor plan name, not Plan A or Plan B)',
  sponsor_name COMMENT 'Employer that sponsors the plan',
  fund_name COMMENT 'Fund',
  asset_class COMMENT 'Fund asset class',
  participants COMMENT 'Number of participants holding the fund',
  total_balance COMMENT 'Sum of balances in US dollars',
  average_balance COMMENT 'Average balance in US dollars',
  as_of_date COMMENT 'Valuation date'
)
COMMENT 'Plan totals by fund for the plans the signed-in user sponsors or advises. Sum
total_balance for plan assets. Empty for participants.'
AS
WITH my_plans AS (
  SELECT DISTINCT scope_id AS plan_id
```

```
  FROM `{{catalog}}`.`{{gov}}`.my_effective_grants
  WHERE scope_type = 'plan' AND can_view_plan_aggregates
)
SELECT t.plan_id, pl.plan_name, pl.sponsor_name, f.fund_name, f.asset_class,
       t.participants, t.total_balance, t.average_balance, t.as_of_date
FROM `{{catalog}}`.`{{base}}`.plan_fund_balances AS t
JOIN my_plans AS m ON m.plan_id = t.plan_id
JOIN `{{catalog}}`.`{{base}}`.plans AS pl ON pl.plan_id = t.plan_id
JOIN `{{catalog}}`.`{{base}}`.funds AS f ON f.fund_id = t.fund_id;

CREATE VIEW `{{catalog}}`.`{{portal}}`.plan_roster (
  participant_id COMMENT 'Participant identifier',
  plan_id COMMENT 'Plan identifier: PLAN-A, PLAN-B or PLAN-C. Users call these Plan A, Plan B and
Plan C.',
  plan_name COMMENT 'Plan name (the sponsor plan name, not Plan A or Plan B)',
  first_name COMMENT 'First name',
  last_name COMMENT 'Last name',
  date_of_birth COMMENT 'Date of birth. NULL when the role of the signed-in user masks it.',
  hire_date COMMENT 'Hire date',
  status COMMENT 'active or terminated',
  total_balance COMMENT 'Total balance across funds in this plan, in US dollars'
)
COMMENT
'Participants of the plans the signed-in user sponsors or advises. Date of birth is masked for
roles whose policy says so.'
AS
WITH roster_plans AS (
  SELECT scope_id AS plan_id, bool_and(mask_dob) AS mask_dob
  FROM `{{catalog}}`.`{{gov}}`.my_effective_grants
  WHERE scope_type = 'plan' AND can_view_roster
  GROUP BY scope_id
)
SELECT p.participant_id, p.plan_id, pl.plan_name, p.first_name, p.last_name,
       CASE WHEN r.mask_dob THEN NULL ELSE p.date_of_birth END,
       p.hire_date, p.status, b.total_balance
FROM `{{catalog}}`.`{{base}}`.participants AS p
JOIN roster_plans AS r ON r.plan_id = p.plan_id
JOIN `{{catalog}}`.`{{base}}`.plans AS pl ON pl.plan_id = p.plan_id
LEFT JOIN `{{catalog}}`.`{{base}}`.participant_plan_balances AS b ON b.participant_id =
p.participant_id AND b.plan_id = p.plan_id;
```

Runtime grants

```
-- What the Northwind portal can do at runtime: read the portal views. Nothing else.
-- Grants go to the application's account group, whose only member is its portal service
principal. Rotate or
-- replace the service principal by changing group membership; no grant or policy changes.
```

```
-- No grants on {{base}} or {{gov}}: the views read those with their owner's rights.
GRANT USE CATALOG ON CATALOG `{{catalog}}` TO `{{portal_group}}`;
GRANT USE SCHEMA ON SCHEMA `{{catalog}}`.`{{portal}}` TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.my_access TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.my_accounts TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.my_contributions TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.plan_summary TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.plan_roster TO `{{portal_group}}`;
GRANT SELECT ON VIEW `{{catalog}}`.`{{portal}}`.fund_lineup TO `{{portal_group}}`;
```

Row policies (layered mode)

Each base table that holds user data carries one ABAC row filter policy, and masked columns a column-mask policy. A table's policy admits exactly the rows some portal view exposes from it, for the signed-in user, at the grain the view exposes them. Balances, for example, admit only the participant's own rows: sponsors and advisors see plan and roster totals, so they read the precomputed `plan_fund_balances` and `participant_plan_balances`. The functions read the claim through `my_effective_grants`, so the claim is still read in one place:

```
-- Row-filter functions for the ABAC row policies (layered enforcement). Unity Catalog applies
them to every base
-- table a portal view reads, for this application's account group only, so pipelines and
administrators are
-- unaffected. Each function admits exactly the rows some portal view exposes from its table, for
the signed-in user.
-- They read the claim through my_effective_grants, which also checks the service principal and
the grant dates.
-- A policy function may read only tables without a policy of their own, so the functions read
governance objects
-- only: the grants, the role policies and the derived participant_plans mapping. Masks follow the
same rules.

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_accounts(p_participant_id STRING)
RETURNS BOOLEAN
COMMENT 'Balances: own accounts only. Sponsors and advisors read precomputed totals instead.'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'participant' AND e.scope_id = p_participant_id AND e.can_view_own_accounts
);

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_contributions(p_participant_id STRING)
RETURNS BOOLEAN
COMMENT 'Contributions: own only. Sponsors and advisors never reach contribution rows.'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'participant' AND e.scope_id = p_participant_id AND e.can_view_own_accounts
);
```

```
CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_roster(p_plan_id STRING)
RETURNS BOOLEAN
COMMENT 'Participants: plans whose roster the signed-in user may list'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'plan' AND e.scope_id = p_plan_id AND e.can_view_roster
);

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_plans(p_plan_id STRING)
RETURNS BOOLEAN
COMMENT 'Plans: plans the signed-in user sponsors or advises, and the plans of their own accounts'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'plan' AND e.scope_id = p_plan_id
) OR EXISTS (
  SELECT 1
  FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  JOIN `{{catalog}}`.`{{gov}}`.participant_plans AS pp ON pp.participant_id = e.scope_id
  WHERE e.scope_type = 'participant' AND e.can_view_own_accounts AND pp.plan_id = p_plan_id
);

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_plan_totals(p_plan_id STRING)
RETURNS BOOLEAN
COMMENT 'Plan totals by fund: plans the signed-in user may total'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'plan' AND e.scope_id = p_plan_id AND e.can_view_plan_aggregates
);

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_roster_totals(p_plan_id STRING)
RETURNS BOOLEAN
COMMENT 'Per-participant totals: plans whose roster the signed-in user may list'
RETURN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'plan' AND e.scope_id = p_plan_id AND e.can_view_roster
);

-- Column mask: a date of birth comes back only when some roster grant for that plan doesn't mask
it, the same rule as
-- plan_roster (the least restrictive grant wins). Rows the user can't reach are already filtered
out.
CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.cm_date_of_birth(p_date_of_birth DATE,
p_plan_id STRING)
RETURNS DATE
COMMENT 'Date of birth, masked unless a roster grant for this plan shows it'
RETURN CASE WHEN EXISTS (
  SELECT 1 FROM `{{catalog}}`.`{{gov}}`.my_effective_grants AS e
  WHERE e.scope_type = 'plan' AND e.scope_id = p_plan_id AND e.can_view_roster AND NOT e.mask_dob
```

```
) THEN p_date_of_birth ELSE NULL END;

CREATE OR REPLACE FUNCTION `{{catalog}}`.`{{gov}}`.rf_deny()
RETURNS BOOLEAN
COMMENT 'Default deny: no rows for tables without a portal grain tag'
RETURN false;
```

The policies are generated from `row_policies` in `example.yml` and a tag set. Each table carries one `portal_grain` value, which selects its policy, and `portal_key` tags bind its columns to the function's arguments:

```
CREATE OR REPLACE POLICY portal_accounts_rows ON SCHEMA `<catalog>`.`northwind_base`
COMMENT 'Portal rows of account_balances for the signed-in user (layered enforcement)'
ROW FILTER `<catalog>`.`northwind_governance`.`rf_accounts`
TO `genie-claims-ref-northwind-portal-backends`
FOR TABLES WHEN has_tag_value('portal_grain', 'accounts')
MATCH COLUMNS has_tag_value('portal_key', 'member') AS member
USING COLUMNS (member)
```

Dates of birth are masked in `participants` itself by `cm_date_of_birth`, with the same rule as the roster view (the least restrictive roster grant for the plan wins), so a probe or an unfiltered view never sees an advisor-masked date. A default-deny policy on the base schema returns no rows to the portal from any table without a grain tag, so a table granted before anyone tags it stays closed. Two Unity Catalog rules shape the layout:

- **A policy function may read only tables that carry no policy of their own** (`UNSUPPORTED_NESTED_ROW_OR_COLUMN_ACCESS_POLICY` otherwise). The governance tables are the policies' inputs, so they stay unprotected and ungranted, and facts the rules need from protected tables are derived into them:

```
-- Derived policy inputs. A row-filter function may read only tables without a policy of their
own, so the facts the
-- rules need are copied into the governance schema. rf_plans uses this to let participants see
the plans their own
-- accounts are in. Rebuilt on every deploy; in production, refresh it with the pipeline that
loads the accounts.
CREATE OR REPLACE TABLE `{{catalog}}`.`{{gov}}`.participant_plans
COMMENT 'Policy input: which plans each participant holds accounts or contributions in. Read only
by row-filter functions.'
AS
SELECT DISTINCT participant_id, plan_id FROM `{{catalog}}`.`{{base}}`.account_balances
UNION
SELECT DISTINCT participant_id, plan_id FROM `{{catalog}}`.`{{base}}`.contributions;

-- Precomputed totals for sponsors and advisors (see 02_base_tables.sql). A row policy can only
admit whole rows, so
-- a role entitled to totals reads totals: it then never reaches the per-fund balances behind
them, even through a
```

```
-- mistaken grant or a probing question.
INSERT OVERWRITE `{{catalog}}`.`{{base}}`.plan_fund_balances
SELECT plan_id, fund_id, COUNT(DISTINCT participant_id), CAST(SUM(balance) AS DECIMAL(16,2)),
       CAST(AVG(balance) AS DECIMAL(14,2)), MAX(as_of_date)
FROM `{{catalog}}`.`{{base}}`.account_balances
GROUP BY plan_id, fund_id;

INSERT OVERWRITE `{{catalog}}`.`{{base}}`.participant_plan_balances
SELECT participant_id, plan_id, CAST(SUM(balance) AS DECIMAL(16,2))
FROM `{{catalog}}`.`{{base}}`.account_balances
GROUP BY participant_id, plan_id;
```

- **A schema-level policy also matches the views in that schema.** Policies attach to the base schema only, which holds no views.

Step 4: Configure the Genie Agent

- **Attach the portal views only.** Never attach base tables, entitlement tables or the policy view.
- **Turn prompt matching off on claim-keyed views.** Prompt matching samples column values with the author's permissions and shares them with every user. Leave it on only for public reference views such as the fund lineup.
- **Compensate in the instructions.** With prompt matching off, Genie can't look up values, so tell it how users name things. Northwind's instructions map "Plan A" to `plan_id = 'PLAN-A'`; without that line Genie filtered on the plan name and found nothing.
- **Queries run as the caller**, which is the default. Every query then carries the user's claim.
- **Can Run for the portal service principal. Can Manage for a small admin group only.** Managers can read every conversation and its stored results.
- **Say how to answer about other people.** Northwind's instructions include: "If a question names a person who does not appear in the data, say that you can't see information about that person. Never answer with the signed-in user's own data instead."

Entitlement pattern catalog

Each pattern below is live in one of the two examples, and the trace cards show what a real token resolved to.

1. Per-user rows

One row per user, role and scope. Onboarding a user is an insert.

```
INSERT INTO <catalog>.northwind_governance.portal_user_grants
VALUES ('pu-1001', 'participant', 'participant', 'P-1001', DATE '2026-01-01', NULL, 'provisioning job', current_timestamp());
```

2. Role policies

What a role may see lives in one row. Turning off `mask_dob` for advisors changes every advisor on their next question.

```
UPDATE <catalog>.northwind_governance.role_policies SET mask_dob = false WHERE role = 'advisor';
```

3. Several roles per user

Linda sponsors Plan A and is a participant in Plan B. Two rows, one session, both views answer:

Linda · Plan sponsor · Plan A, and participant · Plan B · claim `pu-2001`

1. The token carries `custom.claim = pu-2001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
sponsor	plan	PLAN-A	open-ended
participant	participant	P-2001	open-ended

3. The portal views return:
   - `my_accounts` : **4** rows
   - `my_contributions` : **18** rows
   - `plan_summary` : **8** rows
   - `plan_roster` : **59** rows

Two roles, two rows. Plan A totals and roster, plus her own Plan B account. Observed on a live workspace on 2026-10-20.

4. Masking by role

Raj is an advisor. The roster view hides dates of birth because his role policy says so:

```
CASE WHEN r.mask_dob THEN NULL ELSE p.date_of_birth END AS date_of_birth
```

Raj · Advisor · Plans A and C · claim `pu-3001`

1. The token carries `custom.claim = pu-3001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
advisor	plan	PLAN-A	open-ended
advisor	plan	PLAN-C	open-ended

3. The portal views return:
   - `my_accounts`: **0** rows
   - `my_contributions`: **0** rows
   - `plan_summary`: **16** rows
   - `plan_roster`: **99** rows

Plan aggregates and rosters for the plans he advises, with dates of birth masked. Observed on a live workspace on 2026-10-20.

When grants disagree, the least restrictive one wins (`bool_and(mask_dob)` across the user's grants for a plan). Change that to `bool_or` if you want the most restrictive grant to win. In layered mode the same rule also runs in the column-mask policy on `participants.date_of_birth`, so the masked value is all a probe or an unfiltered view can ever see. Change both together.

5. Relationship rules derived from data

Larkspur subscribers whose role includes minor dependents also see their children under 18. No grant rows are needed for the children, and adult dependents stay private:

```
SELECT m.member_id, 'minor dependent' AS visible_as
FROM <catalog>.larkspur_governance.my_effective_grants AS e
JOIN <catalog>.larkspur_base.members AS m ON m.subscriber_id = e.scope_id
```

```
WHERE e.include_minor_dependents AND m.relationship = 'child'
  AND m.date_of_birth > add_months(current_date(), -216)
```

Ana · Subscriber · Birchwood Unified Schools plan · claim `rx-1001`

1. The token carries `custom.claim = rx-1001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
subscriber	member	M-1001	open-ended

3. The portal views return:
   - `my_claims`: **27** rows
   - `my_accumulators`: **2** rows
   - `sponsor_spend_summary`: **0** rows
   - `pharmacy_remittance`: **0** rows

Sees her own claims and her 12-year-old's. Not her spouse's, and not her 19-year-old's. Observed on a live workspace on 2026-10-20.

6. Aggregates with a minimum group size

Benefits administrators see spend by quarter and drug class, never member rows. Groups with fewer distinct members than the role's `min_cell_size` disappear. The totals are precomputed into `sponsor_spend_cells`, so a user's query is never evaluated against the member rows behind a hidden group. Groups smaller than the smallest minimum of any aggregate role are never stored, and the table's row policy applies each role's own minimum:

```
WHERE c.members >= ms.min_cell_size   -- in the view; rf_cells applies the same rule in the row policy
```

A `HAVING` clause over member-level rows is not enough on its own: a probe on the group's own columns can reveal that a hidden group exists. The `theo-cell` side-channel probe tests exactly that.

Dana · Benefits administrator · Alder Manufacturing, and a member · claim `rx-2001`

1. The token carries `custom.claim = rx-2001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
subscriber	member	M-2001	open-ended
sponsor_admin	sponsor	SPN-ALDER	open-ended

3. The portal views return:
   - `my_claims`: **18** rows
   - `my_accumulators`: **1** rows
   - `sponsor_spend_summary`: **21** rows
   - `pharmacy_remittance`: **0** rows

Two roles. Alder spend by quarter and drug class with groups under 11 members hidden, plus her own claims. Observed on a live workspace on 2026-10-20.

Totals then cover the visible groups only, so the Genie instructions tell it to say so.

7. Time-bound grants

Theo's consulting engagement with Cedar Logistics ended on June 30, so that grant no longer matches. His consultant role also uses a larger minimum group size (20) than Dana's administrator role (11), so he sees fewer Alder groups than she does:

Theo · Benefits consultant · Alder and Birchwood (Cedar engagement ended June 30) · claim `rx-3001`

1. The token carries `custom.claim = rx-3001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
consultant	sponsor	SPN-ALDER	open-ended
consultant	sponsor	SPN-BIRCH	open-ended

3. The portal views return:
   - `my_claims` : **0** rows
   - `my_accumulators` : **0** rows
   - `sponsor_spend_summary` : **9** rows
   - `pharmacy_remittance` : **0** rows

Consultant role hides groups under 20 members. His Cedar grant expired, so Cedar returns nothing. Observed on a live workspace on 2026-10-20.

8. Masking identity for a partner

Pharmacy staff reconcile claims filled at their pharmacy. The member id becomes a one-way reference:

```
substr(sha2(concat('larkspur-demo:', c.member_id), 256), 1, 16) AS member_ref   -- in 05_derived_tables.sql
```

Priya · Pharmacy staff · Maple Street Pharmacy · claim `rx-4001`

1. The token carries `custom.claim = rx-4001`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

role	scope_type	scope_id	valid_to
pharmacy	pharmacy	PH-0007	open-ended

3. The portal views return:
   - `my_claims`: **0** rows
   - `my_accumulators`: **0** rows
   - `sponsor_spend_summary`: **0** rows
   - `pharmacy_remittance`: **322** rows

Claims filled at her pharmacy for remittance, with member identity replaced by a one-way reference. Observed on a live workspace on 2026-10-20.

The reference is computed when the remittance rows are built (`pharmacy_remittance_rows`), and the view reads those rows, so pharmacy staff never reach a raw member id, even through a mistaken grant: the claims policy admits members only. A column mask on `member_id` wouldn't work here, because the claims row filter reads that column (`UC_USING_REF_OF_MASKED_COLUMNS`). This hash is demo grade. In production use a keyed hash or a token vault so a reference can't be reversed by hashing every member id.

9. Nothing for unknown users, and one service principal per application

A user who signs in but has no grants sees nothing:

New user · Signed in, no entitlements yet · claim `pu-9999`

1. The token carries `custom.claim = pu-9999`.
2. `my_effective_grants` keeps the grants for this claim and this service principal that are active today, joined to their role policy:

   No active grants. Every portal view returns no rows.

3. The portal views return:
   - `my_accounts`: **0** rows
   - `my_contributions`: **0** rows
   - `plan_summary`: **0** rows
   - `plan_roster`: **0** rows

Authenticated by the app but not in the entitlements table. Sees nothing. Observed on a live workspace on 2026-10-20.

Each application's views also check `session_user()`, so a Larkspur token carrying a Northwind user's claim is denied by grants and, even with a mistaken grant, returns no rows.

Step 5: Integrate your backend

The local app in `app/` is a working reference. The parts that matter in production:

- **Take the user id from your server-side session.** Never from the browser, the chat or a URL. Validate its format before minting: Unity Catalog trims surrounding spaces from the claim and keeps its case, so `" pu-1001 "` matches `pu-1001` and `PU-1001` matches nothing. See `src/claims_kit/claims.py`.
- **Mint and cache one token per user.** `client_credentials` with `custom_claim`, cached until a few minutes before it expires, never sent to the browser. To avoid storing a client secret, use the `urn:ietf:params:oauth:grant-type:token-exchange` grant (workload identity federation) with the same `custom_claim` parameter. See `src/claims_kit/tokens.py`.
- **Bind conversations to users.** Genie conversations belong to the service principal, and any token from that service principal can read them. Keep your own map of user to conversation id, never accept a conversation id from the client, start a new conversation per session, delete it at sign-out, and delete old conversations on a schedule. See `app/conversations.py`.
- **Write your own audit log.** Query history attributes every query to the service principal, so Databricks audit data can't tell you which end user asked. Log the user id, conversation id, message id and statement ids, and join to query history on statement id. See `app/audit.py`.

- **Retry transient warehouse errors** and nothing else. See `StatementRunner.execute`.
- **Trace each request while you build.** `app/ask_flow.py` records every step a question takes, from session to audit record, which makes isolation easy to explain and to debug.

Security model and best practices

- The user id comes only from your authenticated session.
- Treat the service principal credential as a master key: it can mint a token for any user. Keep it in a vault, rotate it, restrict where it can be used, or use workload identity federation and store no secret.
- Keep tokens short-lived and on the server.
- Keep base tables free of claim logic so pipelines and administrators keep working on them.
- Grant the portal service principal `SELECT` on portal views and nothing else. The isolation suite proves it can't read base or governance objects, change entitlements or create objects.
- One service principal per application, in its own account group, and every view checks `session_user()`.
- Row policies on every base table the portal views read, plus default-deny, so a mistake in one layer never exposes rows (see the Admin Runbook for tag governance and guardrail queries).
- Keep prompt matching off on claim-keyed views, and keep Can Manage on the Genie Agent to a small group.
- Run the isolation suite after every change to views, entitlements or the Genie Agent, and keep the results as evidence.

Behavior notes

- **No claim.** Depending on the workspace's current release, `current_oauth_custom_identity_claim()` either returns `NULL` or raises `OAUTH_CUSTOM_IDENTITY_CLAIM_NOT_PROVIDED` when the token carries no claim. The function is evaluated when a statement is analyzed, so in the second case even `CREATE VIEW` needs a claim-carrying session. Notebook 04 detects which case applies.
- **Trimming and case.** Surrounding spaces are trimmed; case is kept.
- **Conversations** belong to the service principal (see Step 5).
- **Attribution.** Query history shows the service principal.
- **Surfaces.** The announcement lists the Genie Conversation API, the SQL Statement Execution API and the JDBC driver. Jobs, notebooks, the SQL editor and dashboards run without a claim, and multi-agent supervisors are not listed, so design for one Genie Agent per request.

Step 6: Validate

`notebooks/08_validate_isolation.py` (task 08 of each deploy job) and `python -m claims_kit.validate --example <name>` mint real tokens for every persona and check each layer:

Category	What it proves
visibility	Each persona sees exactly the ids its grants allow, in every portal view, with no duplicated rows
role_effect	Masking rules from role policies hold
constraint	The entitlements table rejects malformed claims (trailing newline, uppercase, leading space)
attack	No claim, injection strings, wildcards, uppercase and control characters see nothing; an admin session without a claim sees nothing
privilege	The portal service principal can't read base or governance objects, change entitlements or create a view
cross_app	Another application's service principal is denied
binding	Even with a mistaken grant, the view answers only for its own service principal
token	Edited, unsigned and corrupted tokens are rejected
genie	Answers contain the persona's own numbers and nobody else's
setup	The row and mask policies on the base schema match the enforcement model the run assumes
side_channel	Error probes can't reveal values in rows or columns the persona may not see (layered mode). In views mode the same probes are recorded under documented.
documented	Platform behaviors your backend must handle (trimming, conversation ownership); never fails the run

The drills (`python -m claims_kit.drills --example <name>` , or the drills jobs) make six misconfigurations one at a time and show that each persona still sees only what their role may see, while one layer alone returns the whole table or every raw value of a masked column. See Choosing an Enforcement Model for the live results.

Results from the live runs behind this guide:

category	pass	fail	inconclusive or note
setup	1	0	0
visibility	39	0	0
role_effect	2	0	0
constraint	3	0	0
attack	7	0	0
documented	2	0	0
privilege	6	0	0
cross_app	1	0	0
binding	1	0	0
token	4	0	0
side_channel	5	0	0
genie	7	0	0

Northwind, live run on 2026-10-20.

category	pass	fail	inconclusive or note
setup	1	0	0
visibility	44	0	0
role_effect	1	0	0
constraint	3	0	0
attack	7	0	0
documented	2	0	0
privilege	6	0	0
cross_app	1	0	0
binding	1	0	0
token	4	0	0

category	pass	fail	inconclusive or note
side_channel	5	0	0
genie	9	0	0

Larkspur, live run on 2026-10-20.

A case is inconclusive, never passed, when the evidence is ambiguous: Genie doesn't finish or doesn't state the expected number, a transient warehouse error interrupts a check, or an object is hidden instead of explicitly denied. Only a leaked number or a wrong row fails a case, and every result records the reason it passed.

Production checklist

1. One service principal per application; its credential in a vault with rotation, or workload identity federation.
2. User id from the authenticated session only, validated against your canonical format.
3. Tokens minted on the server, cached per user, never sent to the browser.
4. Portal service principal in its application's account group; the group has `USE CATALOG`, `USE SCHEMA` and `SELECT` on portal views only.
5. Claim-keyed views owned by an account group, with explicit column lists and comments on every column.
6. Base tables without claim logic; entitlement tables with CHECK constraints.
7. Governed tags `portal_grain` and `portal_key` with ASSIGN limited to a governance group; one row policy per protected table, default-deny on the base schema, the coverage check passing, tables clustered on the policy keys.
8. Genie Agent on portal views only, prompt matching off on claim-keyed views, Can Manage limited, instructions for naming and for other people.
9. Conversation ids bound to users on the server; deleted at sign-out and on a schedule.
10. Per-user audit log joined to query history on statement id.
11. Transient errors retried; Genie polling with backoff and a timeout.
12. Isolation suite and the guardrail queries run on every change, the drills on a test deployment, with results kept as evidence.

Troubleshooting

Symptom	Cause and fix
Token request returns 403 `custom_claim is not enabled for account`	Custom identity claims are not available for this account yet. Ask your Databricks account team.

Symptom	Cause and fix
`OAUTH_CUSTOM_IDENTITY_CLAIM_NOT_PROVIDED` while creating a view	The workspace evaluates the function without a claim. Create claim-keyed views from a claim-carrying session (notebook 04 does).
The portal service principal gets `INSUFFICIENT_PERMISSIONS` on the warehouse	Grant it Can Use on the warehouse (bootstrap does).
Genie reports "Failed to fetch tables" when creating the agent	The views don't exist yet. Create the agent after the views (task 06 runs after 04 and 05).
`ALTER VIEW ... OWNER TO` is denied	The deploying user needs ownership or MANAGE on the catalog or schema, or metastore admin.
The admin console rejects a grant with a CHECK constraint error	The console checks ids, roles and scope types itself, so this is usually an end date before the start date. Jobs that write grants directly also hit the claim, role and scope constraints.
Statements fail with "catalog not found" for an unqualified name	Fully qualify every name. The workspace's default catalog may not be the one you expect.
Genie answers with zero for a plan or sponsor	It filtered on a display name. Add the naming rule to the instructions (prompt matching is off on claim-keyed views).